Technical Note: N-dimensional Cubic Spline Interpolation

NdimSpline_JAX Project

Contents

Technical Note: Mathematical Theory and Computational Implementation of N-dimensional Cubic Spline Interpolation **1**
Contents . . . 1
1. 1D Natural Cubic Spline . . . 2
2. B-spline Representation . . . 2
3. Coefficient Equations and Tridiagonal System . . . 2
4. Natural Boundary Conditions . . . 2
5. Thomas Algorithm (TDMA) . . . 3
6. N-dimensional Tensor Product Spline . . . 3
7. Efficient Coefficient Computation via Kronecker Factorization . . . 4
8. Localized Evaluation . . . 4
9. Implementation Mapping . . . 5
10. References . . . 5

Technical Note: Mathematical Theory and Computational Implementation of N-dimensional Cubic Spline Interpolation

This document provides a self-contained description of the mathematical theory behind the N-dimensional natural cubic spline interpolation implemented in `ndim_spline_jax`. It covers the 1D formulation, the extension to N dimensions via tensor products, the efficient coefficient computation exploiting Kronecker structure, and the localized evaluation algorithm.

Contents

1. 1D Natural Cubic Spline
2. B-spline Representation
3. Coefficient Equations and Tridiagonal System
4. Natural Boundary Conditions
5. Thomas Algorithm (TDMA)
6. N-dimensional Tensor Product Spline
7. Efficient Coefficient Computation via Kronecker Factorization
8. Localized Evaluation
9. Implementation Mapping
10. References

---

1. 1D Natural Cubic Spline

Consider a scalar function f sampled at $n+1$ equidistant grid points on the interval $[a, b]$:

```
x_k = a + k h, \quad k = 0, 1, \ldots, n, \quad h = \frac{b - a}{n}. \qquad \textrm{(1)}
```

The data values are:

```
y_k = f(x_k), \quad k = 0, 1, \ldots, n. \qquad \textrm{(2)}
```

We seek a piecewise cubic polynomial $s(x)$ satisfying:

- **Interpolation:** $s(x_k) = y_k$ for all $k = 0, \ldots, n$.
- **Smoothness:** $s \in C^2[a, b]$, i.e., s, s', and s'' are all continuous.
- **Natural boundary conditions:** $s''(a) = s''(b) = 0$.

2. B-spline Representation

The interpolant is expressed as a linear combination of cubic B-spline basis functions (de Boor, 1978):

```
s(x) = \sum_{i=0}^{n+2} c_i \, B_i(x), \qquad \textrm{(3)}
```

where c_i are the $n+3$ unknown coefficients. The cubic B-spline basis function $B_i(x)$ is a piecewise cubic with local support, centered at the knot x_{i-1}. In normalized form with $t = |(x - x_{i-1})/h|$, the cardinal B-spline is:

```
\beta(t) = \begin{cases} 4 - 6t^2 + 3t^3 & \text{if } 0 \le t \le 1 \\ (2 - t)^3 & \text{if } 1
```

so that

```
B_i(x) = \beta\!\left(\left|\frac{x - x_{i-1}}{h}\right|\right). \qquad \textrm{(5)}
```

Each B_i is nonzero only on a support of width $4h$ and is C^2 everywhere. Importantly, at any point $x \in [x_k, x_{k+1}]$, at most 4 basis functions are nonzero: B_k, B_{k+1}, B_{k+2}, B_{k+3}.

3. Coefficient Equations and Tridiagonal System

Substituting the interpolation condition $s(x_k) = y_k$ into Eq. (3) and evaluating the B-splines at the grid points (using the values $\beta(0) = 4$, $\beta(1) = 1$, $\beta(2) = 0$):

```
c_{k} + 4 c_{k+1} + c_{k+2} = y_k, \quad k = 0, 1, \ldots, n. \qquad \textrm{(6)}
```

This gives $n+1$ equations for $n+3$ unknowns. The two additional degrees of freedom are fixed by boundary conditions (Section 4). The interior equations ($k = 1, \ldots, n-1$) form the tridiagonal linear system:

```
\underbrace{\begin{pmatrix} 4 & 1 & & \\ 1 & 4 & 1 & \\ & \ddots & \ddots & \ddots \\ & & 1 & 4
```

where c_1 and c_{n+1} are determined by the boundary conditions.

4. Natural Boundary Conditions

The natural spline condition $s''(a) = 0$ and $s''(b) = 0$ translates, via the second derivative of the B-spline expansion, to:

```
c_0 - 2c_1 + c_2 = 0, \qquad \textrm{(8a)}
```

```
c_n - 2c_{n+1} + c_{n+2} = 0. \qquad \textrm{(8b)}
```

From Eq. (6) with $k = 0$: $c_0 + 4c_1 + c_2 = y_0$. Adding this to Eq. (8a) gives $6c_1 = y_0$, hence:

```
c_1 = \frac{y_0}{6}. \qquad \textrm{(9a)}
```

Similarly, from Eq. (6) with $k = n$ and Eq. (8b):

```
c_{n+1} = \frac{y_n}{6}. \qquad \textrm{(9b)}
```

The boundary coefficients are then:

```
c_0 = 2c_1 - c_2, \qquad \textrm{(10a)}
```

```
c_{n+2} = 2c_{n+1} - c_n. \qquad \textrm{(10b)}
```

With c_1 and c_{n+1} known from Eq. (9), the right-hand side of Eq. (7) is fully determined, and the $n - 1$ interior coefficients $c_2, \ldots, c_n$ are obtained by solving the tridiagonal system. Finally, c_0 and c_{n+2} are computed from Eq. (10).

5. Thomas Algorithm (TDMA)

The matrix A in Eq. (7) is symmetric, tridiagonal, and strictly diagonally dominant ($|4| > |1| + |1|$), guaranteeing the existence of a unique solution and numerical stability without pivoting.

The Thomas algorithm (Tridiagonal Matrix Algorithm, TDMA) solves $A\mathbf{x} = \mathbf{d}$ in $O(m)$ operations for an $m \times m$ tridiagonal system. For the specific (1, 4, 1) structure:

Forward sweep ($i = 1, 2, \ldots, m - 1$):

```
w_0 = 4, \quad g_0 = \frac{d_0}{w_0}, \qquad \textrm{(11a)}
```

```
w_i = 4 - \frac{1}{w_{i-1}}, \quad g_i = \frac{d_i - g_{i-1}}{w_i}. \qquad \textrm{(11b)}
```

Backward substitution ($i = m - 2, m - 3, \ldots, 0$):

```
x_{m-1} = g_{m-1}, \qquad \textrm{(12a)}
```

```
x_i = g_i - \frac{x_{i+1}}{w_i}. \qquad \textrm{(12b)}
```

In the implementation (tdma.py), both sweeps are executed with `jax.lax.scan`, which provides a JIT-compatible sequential loop with $O(1)$ memory overhead per step.

6. N-dimensional Tensor Product Spline

For an N-dimensional rectilinear grid with axis-d having n_d intervals and spacing $h_d = (b_d - a_d)/n_d$, the data tensor is:

```
\mathcal{Y}_{k_1 k_2 \cdots k_N} = f(x^{(1)}_{k_1},\, x^{(2)}_{k_2},\, \ldots,\, x^{(N)}_{k_N})
```

The tensor product spline interpolant is:

```
s(\mathbf{x}) = \sum_{i_1=0}^{n_1+2} \cdots \sum_{i_N=0}^{n_N+2} \mathcal{C}_{i_1 \cdots i_N} \
```

where $B^{(d)}_{i_d}$ is the 1D B-spline basis for axis d, and $\mathcal{C}$ is the coefficient tensor of shape $(n_1 + 3) \times \cdots \times (n_N + 3)$.

The interpolation conditions $s(\mathbf{x_k}) = \mathcal{Y}_\mathbf{k}$ at all grid points, together with natural boundary conditions on each axis, yield the global linear system:

```
(A^{(N)} \otimes \cdots \otimes A^{(2)} \otimes A^{(1)}) \, \mathrm{vec}(\mathcal{C}_{\mathrm{i
```

where $\otimes$ denotes the Kronecker product, $A^{(d)}$ is the $(n_d - 1) \times (n_d - 1)$ tridiagonal matrix from Eq. (7) for axis d, $\mathcal{C}_{\text{int}}$ denotes the interior coefficients, and $\mathcal{D}$ is the appropriately modified right-hand side tensor. Solving Eq. (15) directly requires $O(M^{3N})$ operations, where $M = \max_d(n_d)$, which is prohibitive for large N.

7. Efficient Coefficient Computation via Kronecker Factorization

The Kronecker product structure of Eq. (15) allows factorization into N sequential 1D solves (Habermann and Kindermann, 2007). The key identity is:

```
(A^{(N)} \otimes \cdots \otimes A^{(1)})^{-1} = (A^{(N)})^{-1} \otimes \cdots \otimes (A^{(1)})
```

This means the coefficient tensor can be computed iteratively:

```
\mathcal{C}^{(0)} = \mathcal{Y}, \qquad \textrm{(17a)}
```

```
A^{(d)} \, \mathcal{C}^{(d)} = \mathcal{C}^{(d-1)} \quad (d = 1, 2, \ldots, N), \qquad \textrm{
```

```
\mathcal{C} = \mathcal{C}^{(N)}. \qquad \textrm{(17c)}
```

In Eq. (17b), the notation $A^{(d)}$ is used as shorthand for the full 1D spline solve operator along axis d, which includes not only the tridiagonal matrix solve (Eq. (7), (11)–(12)) but also the boundary condition computation (Eq. (9)–(10)). Concretely, for each fixed combination of indices along all axes other than d, extract the 1D vector along axis d, apply the complete 1D solve procedure, and store the result.

Complexity analysis. At step d, the tensor has $\prod_{j \neq d} n_j$ independent 1D problems, each of size $O(n_d)$. The total cost is:

```
\sum_{d=1}^{N} \left(\prod_{j \ne d} n_j\right) \cdot O(n_d) = N \cdot O\!\left(\prod_{d=1}^{N}
```

where we used the approximation $n_d \approx M$ for all d. Compared to $O(M^{3N})$ for direct solve, this is a dramatic reduction.

Implementation (tdma.py, `compute_coefs`): The function iterates `for axis in range(ndim)` and calls `solve_along_axis`, which transposes the target axis to the last position, reshapes into a 2D batch, applies `jax.vmap(solve_1d_spline)` over the batch dimension, and reshapes back. This avoids explicit Python loops over batch indices.

8. Localized Evaluation

Given a query point $\mathbf{x} \in \mathbb{R}^N$, due to the local support of B-splines (Section 2), the sum in Eq. (14) reduces to only 4^N nonzero terms. For each dimension d:

Step 1. Locate the interval: compute the normalized coordinate $u_d = (x_d - a_d)/h_d$ and the interval index:

```
k_d = \mathrm{clip}\!\left(\lfloor u_d \rfloor,\; 0,\; n_d - 1\right). \qquad \textrm{(19)}
```

Step 2. Compute local basis values: define the fractional position $\tau_d = u_d - k_d \in [0, 1)$. The 4 nonzero basis values are:

```
\phi^{(d)}_0 = \beta(|\tau_d + 1|), \quad \phi^{(d)}_1 = \beta(|\tau_d|), \quad \phi^{(d)}_2 =
```

corresponding to coefficient indices $k_d,\ k_d+1,\ k_d+2,\ k_d+3$.

Step 3. Extract local coefficients: use `jax.lax.dynamic_slice` to extract the $4 \times 4 \times \cdots \times 4$ sub-tensor:

```
\mathcal{C}_{\mathrm{local}} = \mathcal{C}[k_1 : k_1\!+\!4, \; k_2 : k_2\!+\!4, \; \ldots, \; k
```

Step 4. Tensor contraction: the interpolated value is:

```
s(\mathbf{x}) = \sum_{j_1=0}^{3} \cdots \sum_{j_N=0}^{3} \mathcal{C}_{\mathrm{local},\, j_1 \co
```

This is implemented as a sequence of tensor-vector contractions:

```
\mathcal{R}^{(0)} = \mathcal{C}_{\mathrm{local}}, \qquad \textrm{(23a)}
```

```
\mathcal{R}^{(d)} = \sum_{j_d=0}^{3} \mathcal{R}^{(d-1)}_{j_d, \ldots} \; \phi^{(d)}_{j_d}, \qu
```

```
s(\mathbf{x}) = \mathcal{R}^{(N)} \in \mathbb{R}. \qquad \textrm{(23c)}
```

Each contraction reduces the tensor rank by one. This is implemented with `jnp.tensordot(result, basis, axes=([0], [0]))`.

Computational cost per evaluation: $O(4^N)$ multiplications, independent of grid size.

Differentiability: Because `dynamic_slice` and all arithmetic operations are JAX primitives, the entire evaluation is compatible with `jax.grad` for automatic differentiation.

9. Implementation Mapping

Theory	Implementation	File
B-spline basis $\beta(t)$, Eq. (4)	`_basis_fn(t)`	interpolant.py
Boundary conditions, Eq. (9)–(10)	`solve_1d_spline(y_data)`	tdma.py
TDMA forward/backward, Eq. (11)–(12)	`tdma_solve(rhs)`	tdma.py
Sequential axis solve, Eq. (17)	`compute_coefs(ndim, y_data)`	tdma.py
Batched 1D solve via `vmap`	`solve_along_axis(data, axis)`	tdma.py
Interval location, Eq. (19)	`_local_index_and_basis(...)`	interpolant.py
Local basis values, Eq. (20)	`_local_index_and_basis(...)`	interpolant.py
Local coefficient extraction, Eq. (21)	`lax.dynamic_slice(c, ...)`	interpolant.py
Tensor contraction, Eq. (22)–(23)	`jnp.tensordot` loop	interpolant.py

10. References

1. C. Habermann and F. Kindermann, "Multidimensional Spline Interpolation: Theory and Applications," *Computational Economics*, vol. 30, no. 2, pp. 153–169, 2007. DOI: 10.1007/s10614-007-9092-4.
2. C. de Boor, *A Practical Guide to Splines*, Springer, 1978.
3. JAX Reference Documentation: https://jax.readthedocs.io/en/latest/.